[책] KNN 분류 알고리즘 이론해설서

문서 버전: v2.1.0.0

이 문서는 KNN 알고리즘의 개념과 원리를 학습하기 위한 이론 참고서입니다.

목차

1. 이 문서에 대하여
2. 분류 문제란?
3. KNN 알고리즘의 원리
4. 거리 측정
5. K값의 선택
6. 결정 경계
7. Train/Test 분할과 모델 평가
8. 혼동행렬과 성능 지표
9. 과적합과 과소적합
10. KNN의 장단점
11. 다른 분류 알고리즘과의 비교
12. 실전 고려사항
13. 용어 사전
14. 더 읽을거리

---

1. 이 문서에 대하여

1.1 목적과 위치

[그림] 공용 01 문서체계도

이 문서는 KNN 분류 시뮬레이터의 배경이 되는 **이론과 수학적 원리**를 설명합니다. 시뮬레이터 화면을 보면서 읽는 것이 아니라, **별도의 학습 시간**에 깊이 있게 공부하기 위한 자료입니다.

문서	깊이	읽는 시점
퀵가이드	[별]	시뮬레이터 사용 중
사용설명서	[별][별][별]	사용 전/후 기능 확인
활용가이드	[별][별][별]	수업 준비 시
이론해설서 (이 문서)	**[별][별][별][별]**	**별도 학습 시간**

1.2 선수 지식

이 문서를 읽기 위해 필요한 수학적 배경은 다음과 같습니다.

수준	필요 지식	해당 섹션
필수	좌표평면, 두 점 사이 거리	§3, §4
권장	제곱근, 절대값	§4
선택	확률·통계 기초 (평균, 분산)	§8, §9

> [팁] 수학 수식이 어렵게 느껴진다면, 각 수식 아래의 **"쉽게 말하면"** 박스를 먼저 읽어보세요.

---

2. 분류 문제란?

2.1 분류의 정의

분류(Classification)란, 주어진 데이터를 미리 정해진 **클래스(범주)** 중 하나에 배정하는 작업입니다.

```
입력: 특성(Feature) 값들      →      분류 모델      →      출력: 클래스 레이블
      (숫자들)                    (알고리즘)                  (범주)
```

2.2 실생활 분류 예시

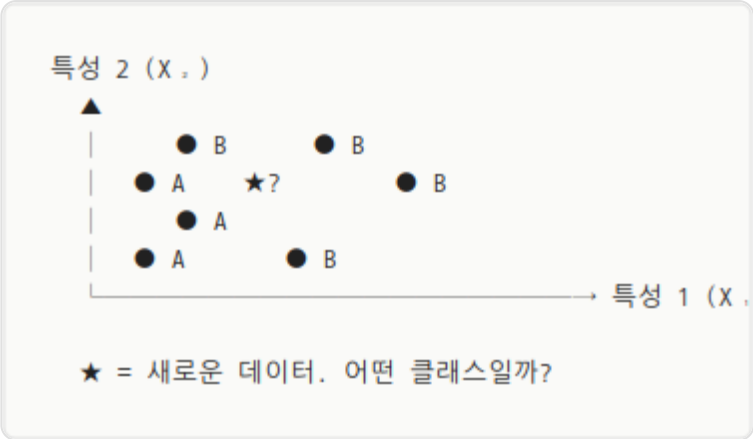
입력 특성	출력 클래스	상황
이메일 내용, 발신자	정상 / 스팸	스팸 필터
증상, 검사 수치	양성 / 악성	의료 진단
키, 무게, 날개 길이	펭귄 종류	생물 분류
픽셀 값	0~9 숫자	손글씨 인식

2.3 분류와 회귀의 차이

항목	분류 (Classification)	회귀 (Regression)
출력 유형	이산적 범주 (A, B, C)	연속적 숫자 (3.14, 98.6)
예시	이 꽃은 장미인가 튤립인가?	이 집의 가격은 얼마인가?
평가 지표	정확도, 혼동행렬	MSE, R^2
KNN 적용	[O] K개 이웃의 다수결 투표	[O] K개 이웃의 평균값

2.4 특성 공간

분류 문제에서 각 데이터는 **특성(Feature)**으로 구성됩니다. 2개의 특성을 사용하면 각 데이터를 2차원 좌표평면 위의 **점**으로 표현할 수 있습니다. 이것이 시뮬레이터의 산점도입니다.



> **쉽게 말하면**: 각 데이터를 지도 위의 점으로 찍는다고 생각하세요. 비슷한 것끼리 모여 있으면, 새 점이 어느 그룹에 가까운지 보고 분류합니다.

---

3. KNN 알고리즘의 원리

3.1 핵심 아이디어

[그림] 이론해설서 01 KNN알고리즘흐름

K-Nearest Neighbors (K-최근접 이웃) 알고리즘의 핵심은 한 문장으로 요약됩니다:

> **"가장 가까운 K개의 이웃이 속한 클래스 중 가장 많은 클래스로 분류한다."**

이것은 일상에서 자연스럽게 사용하는 추론 방식과 같습니다. 새로운 동네에 이사했을 때, 주변 이웃 5명이 모두 학생이라면 그 동네를 "학생 동네"라고 판단하는 것과 같은 원리입니다.

3.2 알고리즘 절차

새로운 점 **q**를 분류하는 과정은 다음과 같습니다.

```
[1단계] 거리 계산
        q와 모든 훈련 데이터 사이의 거리를 계산한다

[2단계] 정렬
        거리를 오름차순으로 정렬한다

[3단계] K개 선택
        가장 가까운 K개의 이웃을 선택한다

[4단계] 다수결 투표
        K개 이웃의 클래스를 집계한다

[5단계] 예측
        가장 많은 표를 받은 클래스를 q의 예측 클래스로 결정한다
```

3.3 수치 예시

문제: 점 q = (5, 4)를 분류하시오. K = 3.

훈련 데이터	좌표	클래스	q와의 거리
d_1	(3, 3)	A	$\sqrt{(5-3)^2+(4-3)^2} = \sqrt{5} \approx 2.24$
d_2	(6, 5)	B	$\sqrt{(5-6)^2+(4-5)^2} = \sqrt{2} \approx 1.41$
d_3	(4, 4)	A	$\sqrt{(5-4)^2+(4-4)^2} = 1.00$
d_4	(7, 3)	B	$\sqrt{(5-7)^2+(4-3)^2} = \sqrt{5} \approx 2.24$
d_5	(6, 4)	B	$\sqrt{(5-6)^2+(4-4)^2} = 1.00$

정렬 결과 (거리 오름차순):

순위	데이터	거리	클래스
1	d_3	1.00	A
2	d_5	1.00	B
3	d_2	1.41	B
4	d_1	2.24	A
5	d_4	2.24	B

K=3일 때 상위 3개: A(1표), B(2표) → **예측: B**

신뢰도: 2/3 = 66.7% (최다 득표 / K)

> **쉽게 말하면**: 가장 가까운 3명에게 물어봤더니 2명이 B라고 했으니, B로 결정합니다. 확신도는 3명 중 2명이니 약 67%입니다.

3.4 게으른 학습 (Lazy Learning)

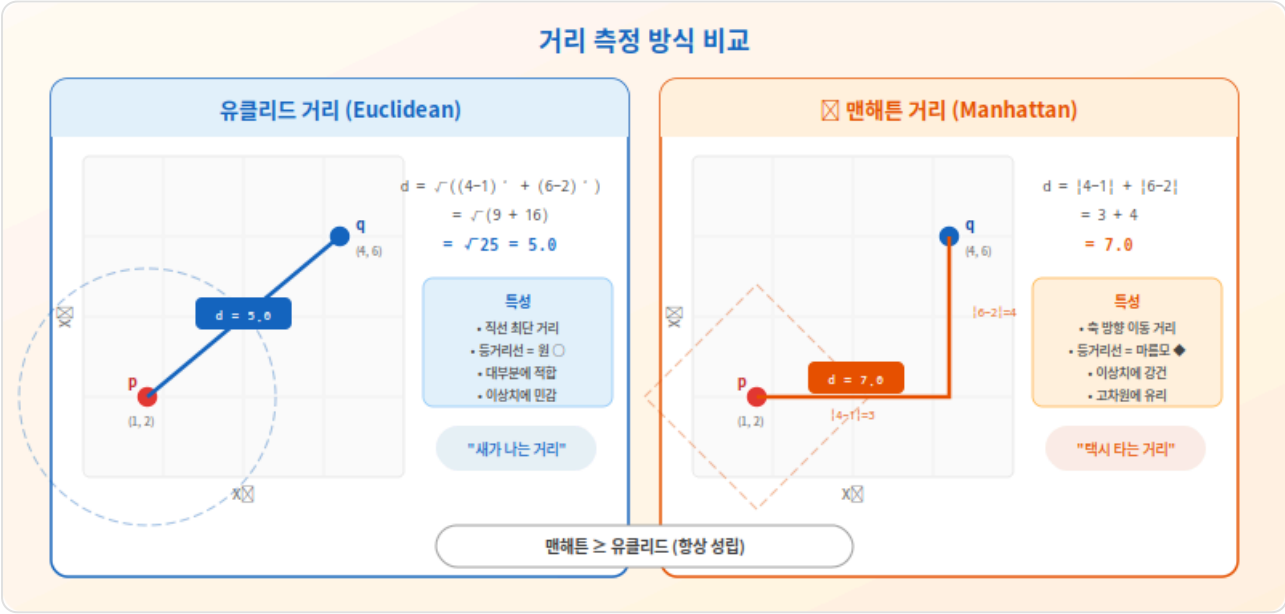
KNN은 **게으른 학습기(Lazy Learner)**로 불립니다. 대부분의 머신러닝 알고리즘은 훈련 데이터에서 모델(규칙, 수식, 트리 등)을 미리 만들어 놓지만, KNN은 훈련 단계에서 아무것도 하지 않습니다. 새 데이터가 들어올 때마다 그제서야 모든 훈련 데이터와의 거리를 계산합니다.

특성	즉시 학습 (Eager)	게으른 학습 (Lazy)
훈련 시간	길다 (모델 구축)	없다 (저장만 함)
예측 시간	빠르다 (모델 적용)	느리다 (전체 거리 계산)
대표 알고리즘	선형 회귀, 의사결정트리, 신경망	**KNN**

---

4. 거리 측정

4.1 왜 거리가 중요한가?



[그림] 이론해설서 02 거리비교

KNN의 "가장 가까운"이라는 판단은 **거리(Distance)** 계산에 의존합니다. 어떤 거리 측정 방식을 사용하느냐에 따라 "가까운 이웃"이 달라지고, 따라서 분류 결과도 달라질 수 있습니다.

4.2 유클리드 거리 (Euclidean Distance)

두 점 사이의 **직선 거리**입니다. 피타고라스 정리를 일반화한 것입니다.

2차원 공식:

$$d(p, q) = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2}$$

n차원 일반화:

$$d(p, q) = \sqrt{\sum_{i=1}^{n} (p_i - q_i)^2}$$

> **쉽게 말하면**: 두 점 사이에 자를 대고 잰 직선 거리입니다. "새가 날아가는 최단 거리"라고도 부릅니다.

기하학적 의미: 유클리드 거리에서 한 점으로부터 같은 거리에 있는 모든 점의 집합은 **원(2차원)** 또는 **구(3차원)**를 이룹니다.

```
    ┌─────────────┐
  ⧈             ⧈
 │       ●       │   ← 반지름 r인 원 위의 모든 점은
 │    (중심)     │     중심에서 거리 r
  ⧈             ⧈
    └─────────────┘
```

4.3 맨해튼 거리 (Manhattan Distance)

축 방향으로만 이동하여 측정하는 거리입니다. 뉴욕 맨해튼의 바둑판 도로에서 유래된 이름입니다.

2차원 공식:

$$d(p, q) = |p_1 - q_1| + |p_2 - q_2|$$

n차원 일반화:

$$d(p, q) = \sum_{i=1}^{n} |p_i - q_i|$$

> **쉽게 말하면**: 도시의 직각 도로를 따라 걷는 거리입니다. 대각선으로 지름길을 갈 수 없고, 오직 가로·세로로만 이동합니다.

기하학적 의미: 맨해튼 거리에서 같은 거리에 있는 점의 집합은 **마름모(다이아몬드)** 형태를 이룹니다.

```
      ◆
    ⧈   ⧈
  ◆   ●   ◆   ← 맨해튼 거리 r인 점들은
    ⧈   ⧈       마름모를 형성
      ◆
```

4.4 유클리드 vs 맨해튼 비교

예시: p = (1, 2), q = (4, 6)

거리 방식	계산	결과
유클리드	$\sqrt{(4-1)^2 + (6-2)^2} = \sqrt{9+16} = \sqrt{25}$	**5.0**
맨해튼	$\lvert 4-1 \rvert + \lvert 6-2 \rvert = 3 + 4$	**7.0**

맨해튼 거리는 항상 유클리드 거리 이상입니다. 이는 삼각부등식에 의해 보장됩니다. 두 거리가 같아지는 유일한 경우는 두 점이 하나의 축 방향으로만 떨어져 있을 때(즉 $p_1=q_1$ 또는 $p_2=q_2$)입니다.

4.5 민코프스키 거리 (일반화)

유클리드와 맨해튼 거리는 사실 **민코프스키 거리(Minkowski Distance)**의 특수한 경우입니다.

```
d(p, q) = ( Σ□□ , ⁿ  |p□ - q□|□ )^(1/p)
```

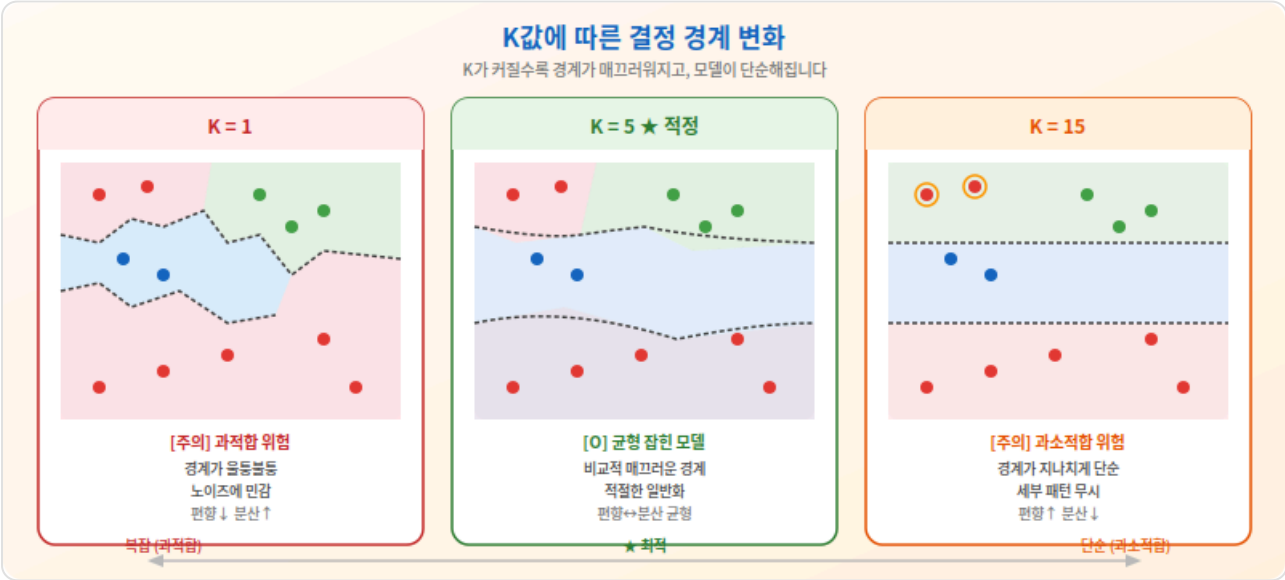
p 값	거리 이름	등거리선 모양
p = 1	맨해튼 거리	마름모 ◆
p = 2	유클리드 거리	원 ○
p → ∞	체비셰프 거리	정사각형 □

4.6 거리 선택 기준

상황	추천 거리	이유
일반적인 경우	유클리드	가장 직관적, 대부분에 적합
특성 간 독립성 가정	맨해튼	축별로 따로 기여
이상치(Outlier)가 많을 때	맨해튼	제곱하지 않아 이상치에 덜 민감
고차원 데이터	맨해튼	차원의 저주에서 상대적으로 강건

5. K값의 선택

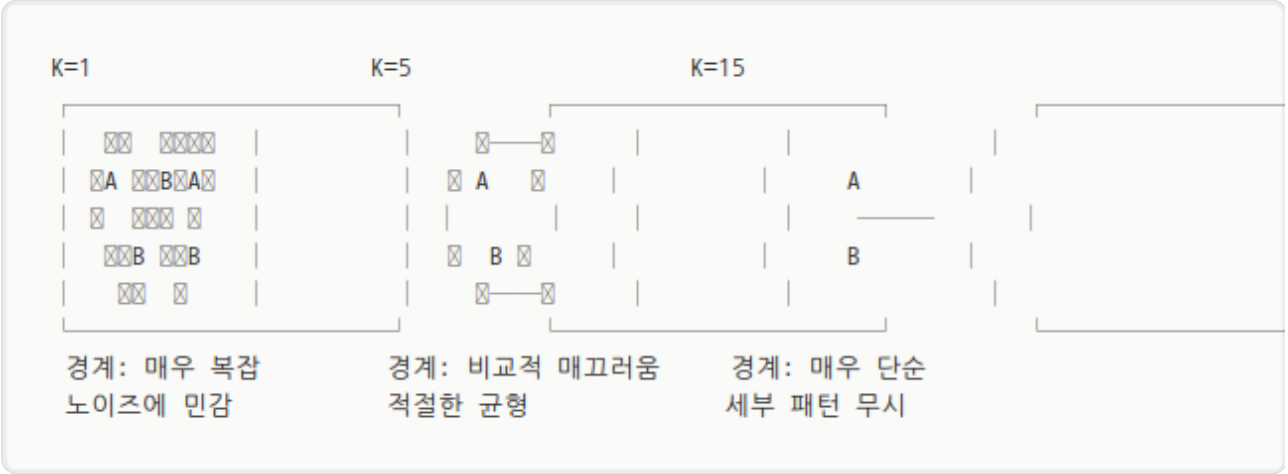
5.1 K값이란?



[그림] 이론해설서 03 K값결정경계

K값은 분류 시 참조할 이웃의 수를 결정하는 **하이퍼파라미터(Hyperparameter)**입니다. 하이퍼파라미터란, 알고리즘이 데이터에서 자동으로 학습하는 것이 아니라 사람이 직접 정해주어야 하는 설정값입니다.

5.2 K값에 따른 변화



K값	편향 (Bias)	분산 (Variance)	결정 경계	위험
작음 (1~3)	낮음	높음	복잡, 울퉁불퉁	과적합
중간 (5~9)	중간	중간	적절히 매끄러움	—
큼 (11~15)	높음	낮음	매우 매끄러움	과소적합

5.3 K=1의 특수성

K=1일 때, KNN은 단순히 "가장 가까운 1개의 데이터와 같은 클래스"로 분류합니다. 이것은 **보로노이 다이어그램 (Voronoi Diagram)**의 분할과 동일합니다. 각 훈련 데이터가 하나의 "영역"을 지배하며, 그 영역 안의 모든 점은 해당 훈련 데이터의 클래스로 분류됩니다.

K=1의 훈련 데이터 정확도는 항상 100%입니다. 각 훈련 데이터의 가장 가까운 이웃은 자기 자신이기 때문입니다. 하지만 이것이 좋은 모델이라는 의미는 아닙니다 — 이것이 바로 Train/Test 분할이 필요한 이유입니다.

5.4 홀수 K를 권장하는 이유

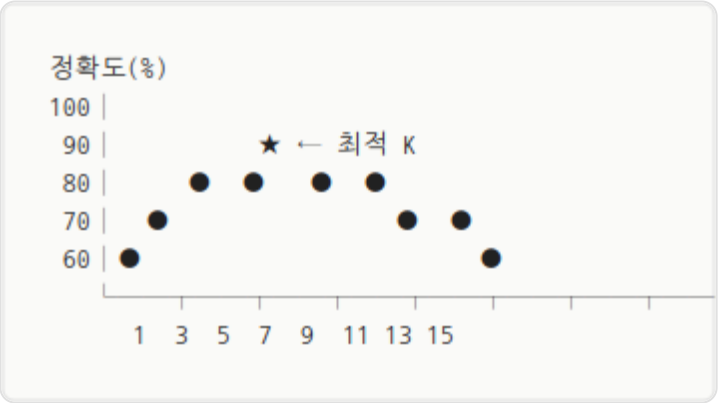
2클래스 분류에서 짝수 K를 사용하면 **동점(Tie)**이 발생할 수 있습니다.

```
K=4일 때: A에 2표, B에 2표 → 동점!
          → 어떻게 결정할 것인가?
```

동점 처리 전략은 다양하지만 (가장 가까운 이웃 우선, 무작위 선택 등), 이런 상황 자체를 피하기 위해 **홀수 K**를 사용하는 것이 관례입니다. 다만 3클래스 이상에서는 홀수여도 동점이 발생할 수 있습니다 (예: K=3에서 각 클래스 1표).

5.5 최적 K 탐색 방법

최적 K를 찾는 일반적인 방법은 여러 K값에 대해 **테스트 정확도**를 측정한 뒤, 가장 높은 정확도를 보이는 K를 선택하는 것입니다.



시뮬레이터에서는 K=1~15(홀수)에 대한 테스트 정확도를 **한 번에 계산**하여 테이블과 차트로 보여줍니다. [트로피] 표시가 최적 K입니다.

> [주의] **주의**: 분류를 반복 실행하면 무작위 셔플로 Train/Test 구성이 달라지므로, 최적 K가 매번 달라질 수 있습니다. 이것은 오류가 아니라 데이터 분할의 무작위성에 의한 자연스러운 변동입니다.

---

6. 결정 경계

6.1 결정 경계란?

결정 경계(Decision Boundary)는 특성 공간에서 서로 다른 클래스의 예측 영역을 나누는 **경계선(또는 경계면)**입니다. 경계의 한쪽에 놓인 점은 클래스 A로, 다른 쪽에 놓인 점은 클래스 B로 분류됩니다.

6.2 KNN의 결정 경계 특성

KNN의 결정 경계는 다른 알고리즘과 달리 **수식으로 표현할 수 없습니다.** 선형 회귀의 결정 경계는 직선(또는 평면)이고, 의사결정트리의 경계는 축에 평행한 직선의 조합이지만, KNN의 경계는 데이터의 분포에 따라 **임의의 형태**를 가집니다.

알고리즘	결정 경계 형태	수식 표현
선형 분류기	직선/평면	[O]
의사결정트리	축 평행 직사각형	[O]
KNN	**임의의 곡선**	**[X]**
신경망	임의의 곡선	[X]

6.3 결정 경계의 계산 방법

시뮬레이터에서는 결정 경계를 다음과 같이 근사합니다.

```
[1] 시각화 영역을 50×50 = 2,500개 격자로 나눔
[2] 각 격자점에서 KNN 분류 수행 (훈련 데이터 기준)
[3] 격자점의 예측 클래스에 해당하는 색상으로 칠함
[4] 색상이 바뀌는 경계 = 결정 경계
```

이 방법은 정확한 경계가 아닌 **근사**이며, 격자 해상도(50×50)에 의해 세부적인 경계 모양이 약간 달라질 수 있습니다.

6.4 K값에 따른 경계 변화의 직관

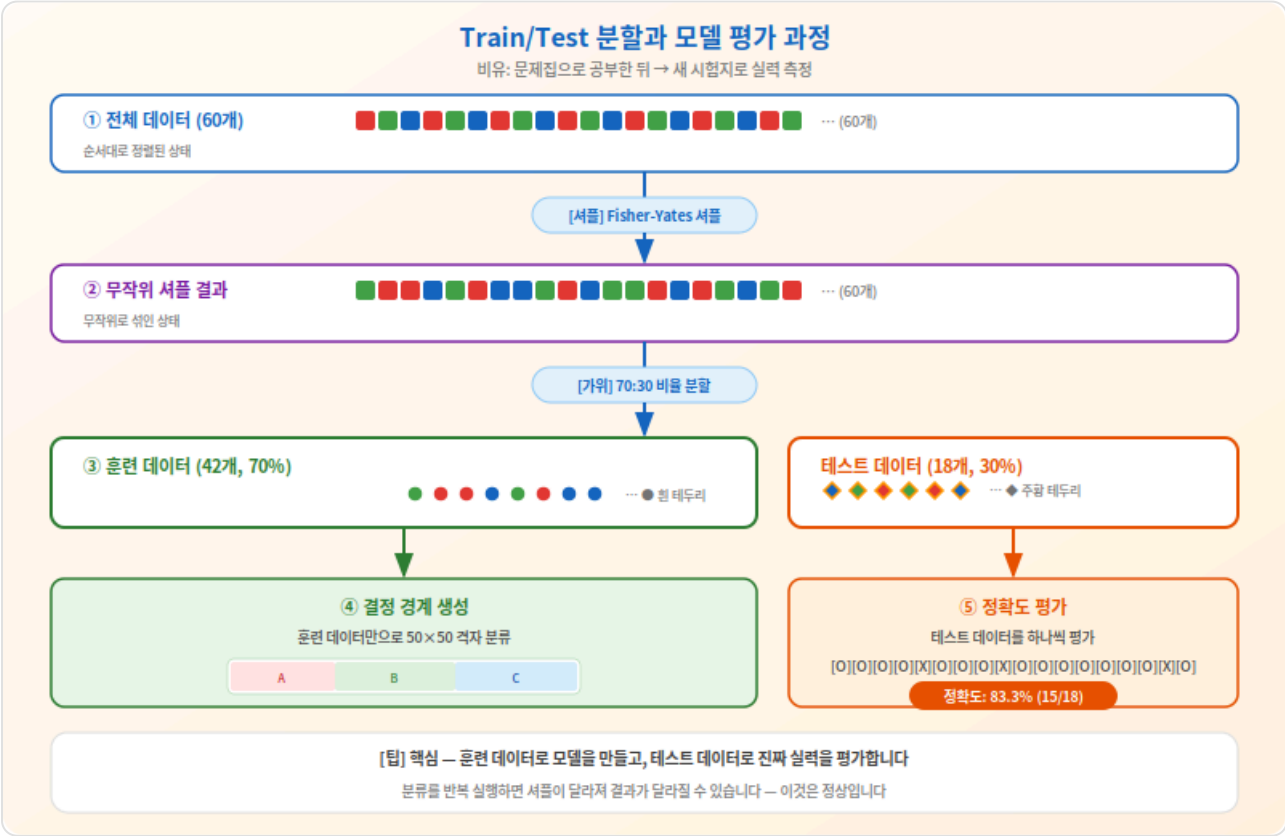
K가 커질수록 결정 경계가 매끄러워지는 이유를 직관적으로 이해해 봅시다.

K=1일 때, 경계 근처의 한 점이 이동하면 가장 가까운 이웃 1개가 바뀔 수 있고, 그러면 예측이 즉시 뒤집힙니다. 경계가 각 데이터 점 사이사이에서 예민하게 반응하므로 울퉁불퉁합니다.

K=15일 때, 같은 점이 이동해도 15개 이웃 중 1개만 바뀌므로 다수결 결과가 쉽게 뒤집히지 않습니다. 결과적으로 경계가 넓은 영역의 "평균적인 경향"을 반영하여 부드러워집니다.

7. Train/Test 분할과 모델 평가

7.1 왜 데이터를 나누는가?



[그림] 이론해설서 05 TrainTest분할

모델의 진정한 가치는 **처음 보는 데이터**에서 얼마나 잘 작동하느냐에 달려 있습니다. 훈련에 사용한 데이터로 평가하면, 모델이 데이터를 "외운" 것인지 "이해한" 것인지 구분할 수 없습니다.

```
비유: 시험 공부

[X] 시험 문제를 미리 보고 외운 뒤 → 같은 문제로 시험 → 100점 (의미 없음)
[O] 문제집으로 공부한 뒤 → 새로운 시험지로 시험 → 진짜 실력 측정
```

용어	역할	비유
훈련 데이터 (Train)	모델 학습에 사용	문제집
테스트 데이터 (Test)	모델 평가에 사용	시험지

7.2 분할 절차

```
전체 데이터 (N개)
        |
   [무작위 셔플]  ← Fisher-Yates 알고리즘
        |
    ┌──────────┴──────────┐
    |                     |
훈련 데이터          테스트 데이터
(N × r개)          (N × (1-r)개)
    |                     |
    ▼                     ▼
결정 경계 생성        정확도 평가
```

Fisher-Yates 셔플: 배열의 마지막 원소부터 시작하여, 남은 원소 중 무작위로 하나를 골라 현재 원소와 교환합니다. 이 방법은 모든 순열이 동일한 확률로 나타나는 것을 보장합니다.

7.3 분할 비율의 트레이드오프

훈련 비율	훈련 데이터	테스트 데이터	모델 성능	평가 신뢰도
60%	적음	많음	↓	↑
70%	적절	적절	↔	↔
80%	많음	적음	↑	↓

> **쉽게 말하면**: 문제집이 두꺼울수록 공부는 잘 되지만, 시험 문제가 너무 적으면 운에 의해 점수가 달라집니다. 70:30이 가장 균형 잡힌 선택입니다.

7.4 교차 검증 (Cross-Validation)

Train/Test 분할의 한계는 **한 번의 분할 결과에 의존**한다는 것입니다. 분할이 달라지면 결과도 달라집니다. 이를 보완하는 방법이 **K-Fold 교차 검증**입니다.

```
5-Fold 교차 검증:

Fold 1: [Test] [Train] [Train] [Train] [Train]  → 정확도₁
Fold 2: [Train] [Test] [Train] [Train] [Train]  → 정확도₂
Fold 3: [Train] [Train] [Test] [Train] [Train]  → 정확도₃
Fold 4: [Train] [Train] [Train] [Test] [Train]  → 정확도₄
Fold 5: [Train] [Train] [Train] [Train] [Test]  → 정확도₅

최종 정확도 = (정확도₁ + ... + 정확도₅) / 5
```

> [팁] 현재 시뮬레이터는 단순 Train/Test 분할만 지원합니다. K-Fold 교차 검증은 향후 기능으로 계획되어 있습니다.

---

8. 혼동행렬과 성능 지표

8.1 혼동행렬 구조

[그림] 이론해설서 06 혼동행렬

혼동행렬(Confusion Matrix)은 분류 결과를 실제 클래스와 예측 클래스의 교차표로 정리한 것입니다.

2클래스 혼동행렬:

```
                    예측
              Positive  Negative
실제 Positive [  TP    ¦   FN   ]
     Negative [  FP    ¦   TN   ]
```

약어	의미	설명
TP	True Positive	양성을 양성으로 맞힘 [O]
TN	True Negative	음성을 음성으로 맞힘 [O]
FP	False Positive	음성을 양성으로 틀림 [X] (1종 오류)
FN	False Negative	양성을 음성으로 틀림 [X] (2종 오류)

다중 클래스 혼동행렬 (3클래스 예시):

```
              예측
          A    B    C
실제 A [ 8    1    0 ]   ← A를 9번 판단: 8번 맞힘, 1번 B로 틀림
     B [ 2    7    1 ]   ← B를 10번 판단: 7번 맞힘, 2번 A로, 1번 C로 틀림
     C [ 0    0    6 ]   ← C를 6번 판단: 모두 맞힘
```

대각선 합(8+7+6=21)이 정분류, 나머지(2+1+1=4)가 오분류입니다.

8.2 정확도 (Accuracy)

```
정확도 = 정분류 수 / 전체 데이터 수
       = (TP + TN) / (TP + TN + FP + FN)
```

다중 클래스에서는 대각선 합 / 전체 합입니다.

위 예시: 21 / 25 = **84.0%**

8.3 정확도의 함정

정확도는 직관적이지만, **클래스 불균형**이 있을 때 오해를 줄 수 있습니다.

```
예: 유방암 진단 — 전체 1000명 중 양성 10명, 음성 990명

"모두 음성"이라고 예측하면:
정확도 = 990/1000 = 99.0%  ← 높은 정확도!
하지만 양성 10명을 모두 놓침 → 의미 없는 모델
```

이런 경우 정밀도(Precision), 재현율(Recall), F1-Score 등 추가 지표가 필요합니다. 이 시뮬레이터에서는 전체 정확도와 혼동행렬을 제공하여, 어떤 클래스에서 오분류가 집중되는지 확인할 수 있습니다.

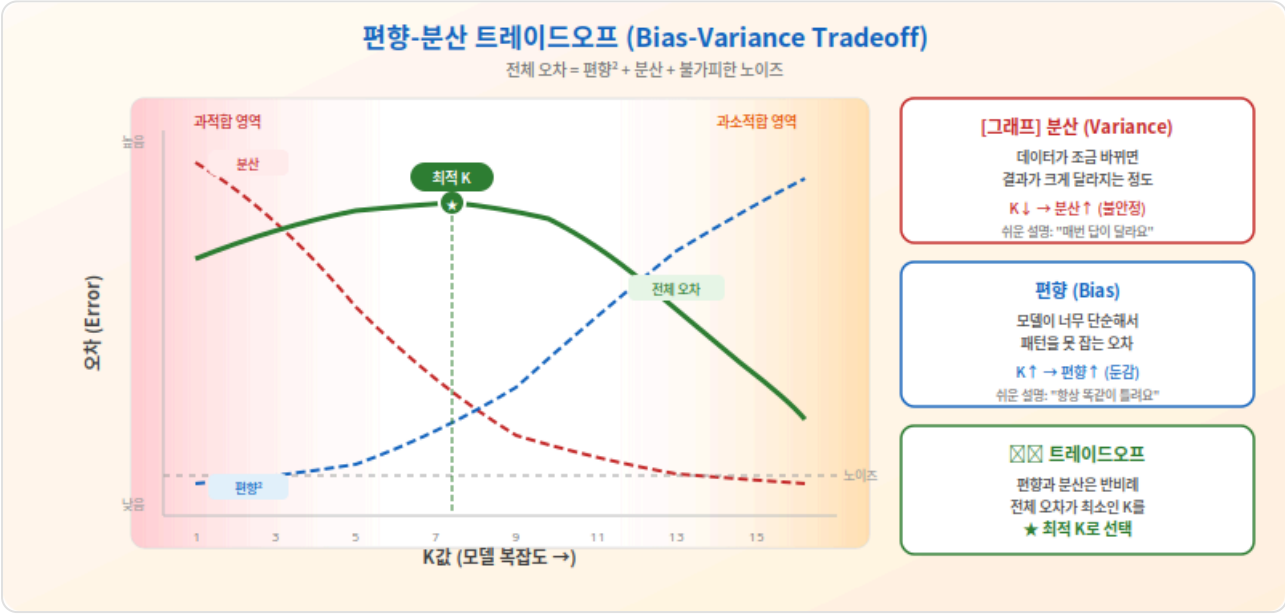
8.4 혼동행렬 읽기 팁

패턴	의미	시뮬레이터에서 확인
대각선 숫자가 크다	전반적으로 잘 분류	높은 정확도
특정 칸의 숫자가 크다	두 클래스가 혼동됨	산점도에서 해당 클래스 영역 겹침
한 행이 여러 열에 분산	해당 클래스 분류 어려움	데이터 분포가 넓거나 다른 클래스와 겹침

---

9. 과적합과 과소적합

9.1 편향-분산 트레이드오프

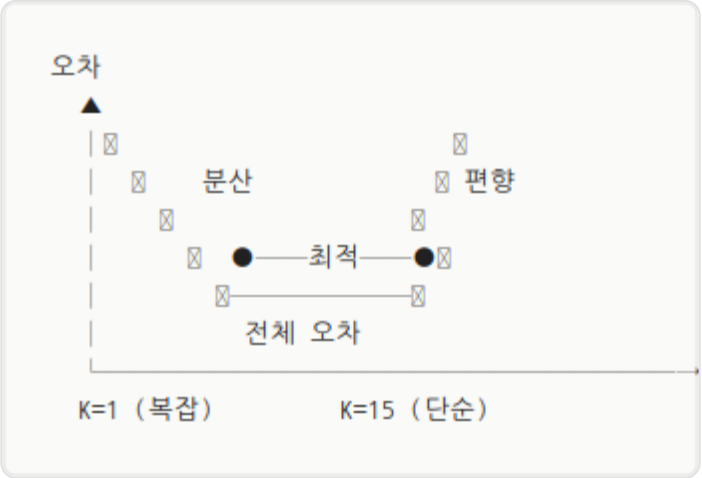


[그림] 이론해설서 04 편향분산트레이드오프

모든 머신러닝 모델의 오차는 세 가지 요소로 분해됩니다.

```
전체 오차 = 편향² + 분산 + 불가피한 노이즈
```

요소	의미	KNN에서
편향 (Bias)	모델이 너무 단순해서 패턴을 못 잡는 오차	K가 클수록 증가
분산 (Variance)	데이터가 조금 바뀌면 결과가 크게 바뀌는 불안정성	K가 작을수록 증가
노이즈	데이터 자체의 랜덤 변동	줄일 수 없음



9.2 과적합 (Overfitting)

과적합은 모델이 훈련 데이터에 **너무 잘** 맞추어 새 데이터에서 성능이 떨어지는 현상입니다.

```
과적합의 신호:
  훈련 정확도: 98%  ← 매우 높음
  테스트 정확도: 65% ← 크게 낮음
  차이: 33%p        ← 큰 격차 = 과적합!
```

KNN에서 과적합은 주로 **K가 작을 때** 발생합니다. K=1이면 가장 가까운 점 하나에 의존하므로, 노이즈 데이터 하나가 경계 전체를 왜곡합니다.

> **쉽게 말하면**: 시험 문제를 달달 외웠지만 정작 응용 문제는 못 푸는 학생과 같습니다. 문제집(훈련)은 100점이지만 시험(테스트)은 65점입니다.

9.3 과소적합 (Underfitting)

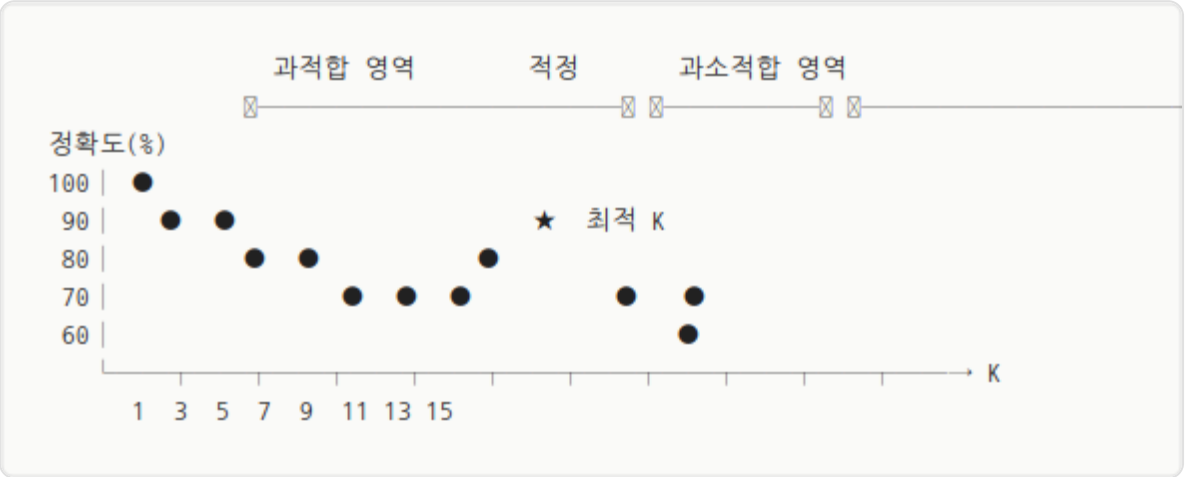
과소적합은 모델이 훈련 데이터의 패턴조차 제대로 포착하지 못하는 현상입니다.

```
과소적합의 신호:
  훈련 정확도: 60%  ← 낮음
  테스트 정확도: 55% ← 비슷하게 낮음
  차이: 5%p         ← 작은 격차, 하지만 둘 다 낮음!
```

KNN에서 과소적합은 **K가 매우 클 때** 발생합니다. K=N(데이터 전체)이면 항상 다수 클래스로 예측하는, 사실상 쓸모없는 모델이 됩니다.

> **쉽게 말하면**: 공부를 너무 대충 해서 문제집도, 시험도 못 푸는 학생입니다.

9.4 적정점 찾기



최적 K는 **테스트 정확도가 가장 높은 K**입니다. 이 지점이 편향과 분산의 최적 균형점입니다. 시뮬레이터의 K값 재생과 정확도 테이블은 이 최적점을 시각적으로 탐색하는 도구입니다.

---

10. KNN의 장단점

10.1 장점

장점	설명
단순성	개념이 직관적이고 구현이 간단
비모수적	데이터 분포에 대한 가정이 없음. 선형이든 비선형이든 적용 가능
적응적 경계	데이터 분포를 그대로 반영하는 임의의 결정 경계 형성
훈련 불필요	새 데이터 추가가 즉시 반영됨 (모델 재훈련 불필요)
다중 클래스	별도 확장 없이 자연스럽게 다중 클래스 분류 가능

10.2 단점

단점	설명	대응 방안
느린 예측	매 예측마다 전체 훈련 데이터와 거리 계산	KD-트리, Ball 트리 등 인덱싱 구조
메모리	모든 훈련 데이터를 저장해야 함	데이터 축소 기법
차원의 저주	고차원에서 거리 개념이 무의미해짐	차원 축소 (PCA 등)
스케일 민감	범위가 큰 특성이 거리를 지배	특성 정규화 필수
불균형 데이터	다수 클래스에 편향된 예측	가중치 KNN, 리샘플링

10.3 차원의 저주

차원의 저주(Curse of Dimensionality)는 특성의 수가 많아질수록 발생하는 문제입니다. 고차원 공간에서는 모든 데이터 점 간의 거리가 비슷해지면서, "가까운 이웃"과 "먼 이웃"의 구분이 무의미해집니다.

```
차원    최근접 이웃 거리 / 최원 이웃 거리
  2        0.3  ← 가까운 것과 먼 것의 차이가 큼
  10       0.6
  50       0.85
  100      0.95 ← 거의 모든 점이 비슷한 거리 → KNN 무의미
```

이것이 시뮬레이터가 **2개 특성(2차원)**만 사용하는 이유 중 하나입니다. 실전에서 고차원 데이터에 KNN을 적용하려면 먼저 **차원 축소**(PCA, t-SNE 등)를 수행하는 것이 일반적입니다.

10.4 특성 정규화의 중요성

특성의 스케일이 다르면, 범위가 큰 특성이 거리를 지배합니다.

```
예: 키(cm)와 체중(kg)으로 분류

  사람 A: (170cm, 65kg)
  사람 B: (175cm, 70kg)

  정규화 없이: d = √((175-170)² + (70-65)²) = √(25+25) = 7.07
                → 키와 체중이 동일하게 기여

  만약 키를 mm로 바꾸면:
  사람 A: (1700mm, 65kg)
  사람 B: (1750mm, 70kg)

  정규화 없이: d = √((1750-1700)² + (70-65)²) = √(2500+25) = 50.2
                → 키가 거리의 99.5%를 지배! 체중은 사실상 무시됨
```

시뮬레이터의 실 데이터셋은 내부적으로 0~100 스케일링을 적용하여 이 문제를 완화합니다.

---

11. 다른 분류 알고리즘과의 비교

항목	KNN	의사결정트리	로지스틱 회귀	신경망
학습 방식	게으른 (저장)	즉시 (트리 구축)	즉시 (가중치 학습)	즉시 (가중치 학습)
결정 경계	임의 곡선	축 평행 직각	직선/평면	임의 곡선
해석 가능성	높음 (이웃 확인)	매우 높음 (규칙)	높음 (계수)	낮음 (블랙박스)
훈련 속도	없음	중간	빠름	느림
예측 속도	느림	매우 빠름	매우 빠름	빠름
비선형 패턴	[O] 잘 잡음	[O] 잘 잡음	[X] 제한적	[O] 매우 잘 잡음
데이터 양 요구	중간	적음~중간	적음	많음
하이퍼파라미터	K, 거리 방식	최대 깊이, 분할 기준	정규화 강도	층수, 노드수, 학습률 등

> **언제 KNN을 선택할까?** 데이터가 적당한 크기(수천~수만)이고, 비선형 패턴이 있으며, 모델의 예측 근거를 직관적으로 확인하고 싶을 때 KNN은 좋은 출발점입니다.

---

12. 실전 고려사항

12.1 KNN을 적용하기 전 체크리스트

#	항목	확인
1	특성을 정규화(또는 표준화)했는가?	☐
2	불필요하거나 상관 높은 특성을 제거했는가?	☐
3	데이터에 결측치가 없는가? (KNN은 결측치 처리 불가)	☐
4	클래스 분포가 심하게 불균형하지 않은가?	☐
5	특성 수가 데이터 수에 비해 너무 많지 않은가? (차원의 저주)	☐

12.2 KNN의 변형

변형	아이디어	장점
가중치 KNN	가까운 이웃에 더 큰 투표 가중치 (1/d)	거리 반영, 동점 해소
반경 기반	고정 반경 내 이웃 사용 (K 대신 r)	밀도에 적응
KD-트리 KNN	공간 분할 트리로 이웃 탐색 가속	O(N) → O(log N)
Approximate KNN	근사 이웃 탐색 (LSH 등)	대규모 데이터 처리

12.3 KNN의 실세계 응용

분야	응용	왜 KNN이 적합한가
추천 시스템	유사 사용자 기반 추천	"비슷한 취향의 사람들이 좋아한 것" = KNN의 핵심 아이디어
이상 탐지	정상 데이터와 거리가 먼 점 탐지	K개 이웃과의 평균 거리로 이상 정도 측정
결측치 대체	유사 데이터의 값으로 채움	K개 이웃의 평균/최빈값으로 결측치 추정
의료 진단	유사 환자 기반 진단 보조	근거(이웃 환자)를 직접 제시할 수 있어 해석 용이

13. 용어 사전

용어	영어	정의
KNN	K-Nearest Neighbors	K개의 가장 가까운 이웃으로 분류하는 알고리즘
K값	K (hyperparameter)	참조할 이웃의 수. 사람이 직접 정하는 하이퍼파라미터
특성	Feature	데이터를 설명하는 측정 가능한 속성. 예: 키, 무게
클래스	Class / Label	분류의 대상이 되는 범주. 예: A, B, C
결정 경계	Decision Boundary	서로 다른 클래스의 예측 영역을 나누는 경계
유클리드 거리	Euclidean Distance	두 점 사이의 직선 거리
맨해튼 거리	Manhattan Distance	축 방향으로만 이동하는 거리
과적합	Overfitting	훈련 데이터에 과도하게 맞춰 일반화 성능이 떨어지는 현상
과소적합	Underfitting	모델이 너무 단순해서 패턴을 제대로 학습하지 못하는 현상
편향	Bias	모델의 가정이 너무 강해서 생기는 체계적 오차
분산	Variance	데이터 변화에 따른 모델 예측의 변동성
혼동행렬	Confusion Matrix	실제 클래스와 예측 클래스의 교차표
정확도	Accuracy	전체 데이터 중 올바르게 분류한 비율
정규화	Normalization	특성의 스케일을 통일하는 전처리
차원의 저주	Curse of Dimensionality	고차원에서 거리 개념이 무의미해지는 현상
게으른 학습	Lazy Learning	훈련 단계에서 모델을 만들지 않고 예측 시 계산하는 방식
하이퍼파라미터	Hyperparameter	알고리즘 외부에서 사람이 정하는 설정값
교차 검증	Cross-Validation	데이터를 여러 번 분할하여 평가의 신뢰도를 높이는 기법
보로노이 다이어그램	Voronoi Diagram	K=1 KNN의 결정 경계와 동일한 공간 분할 구조

14. 더 읽을거리

14.1 입문 자료

#	자료	설명
1	Cover, T. & Hart, P. (1967). "Nearest neighbor pattern classification."	KNN의 이론적 기반을 제시한 고전 논문
2	James, G. et al. (2021). *An Introduction to Statistical Learning (ISLR)* Ch.4	KNN을 포함한 분류 방법 입문 (무료 PDF)
3	Hastie, T. et al. (2009). *The Elements of Statistical Learning (ESL)* Ch.13	프로토타입과 최근접 이웃 심화

14.2 시뮬레이터에 사용된 데이터셋

데이터셋	원전
Iris	Fisher, R.A. (1936). "The use of multiple measurements in taxonomic problems."
Palmer Penguins	Horst, A.M. et al. (2020). palmerpenguins R package
Wine	Aeberhard, S. et al. (1991). UCI Machine Learning Repository
Seeds	Charytanowicz, M. et al. (2012). "Complete gradient clustering algorithm."
Breast Cancer	Wolberg, W.H. et al. (1995). UCI Machine Learning Repository

14.3 관련 프로젝트 문서

문서	내용
퀵가이드 v2.1.0.0	시뮬레이터 조작법 빠른 참조
사용설명서 v2.1.0.0	모든 기능 상세 설명
활용가이드 v2.1.0.0	교수자용 수업 적용 안내
개발일지 v2.1.0.0	기술 구현 세부사항

생성/수정 이력

버전	날짜	변경 내용	작성자
v2.1.0.0	2026-02-15	초안 작성 — 분류 문제 개념, KNN 원리(수치 예시 포함), 거리 측정(유클리드/맨해튼/민코프스키), K값 선택(편향-분산 트레이드오프), 결정 경계, Train/Test 분할, 혼동행렬과 성능 지표, 과적합/과소적합, 장단점, 알고리즘 비교, 실전 고려사항, 용어 사전 53항	Claude

---

KNN 분류 알고리즘 이론해설서 — 별도 학습 시간에 읽는 개념과 원리 참고서